Protecting the ALU Logo on the Ethereum Blockchain

- GitHub repository: https://github.com/devark28/alu-dapp-proj_1_assgmnt
- Demo video: https://youtu.be/KaoQenSY1VY

Abstract

This project registers the official African Leadership University logo on Ethereum so that its ownership and exact file contents are recorded permanently and can be checked by anyone. An ERC-721 contract stores the logo's SHA-256 fingerprint, the registering wallet and the time of registration, and tells any visitor whether a logo file they hold is the authentic one. An ERC-20 contract then splits ownership of the logo into 1,000,000 shares that ALU can distribute to its community.

Introduction

ALU's logo is the visual signature of the university. It appears on certificates, websites and partner material, and an altered or falsely claimed copy can mislead students, employers and partners. Today the proof of what the "real" logo looks like sits in places ALU controls privately: a brand folder, a design file, a database. Those records can be changed by anyone with access, and an outsider has no way to confirm that what they are shown is the original.

A database answers "what does ALU's server say the logo is", and only for as long as the server is trusted and running. A copyright registration proves a legal claim, but it is slow, national, and does not let anyone check a specific file instantly. A public blockchain gives a third option: a record that ALU cannot quietly rewrite, that carries a timestamp nobody can backdate, and that any person in any country can query for free. Storing the logo's SHA-256 hash rather than the image itself keeps the record small while still tying it to every byte of the file.

This work connects directly to my mission: to tackle location data infrastructure at scale, driven by growing urbanisation and increasing business use cases, through a multi-tenant platform that applications use to handle their users' location data at scale and to comply with location-related regulations. The same building blocks used here, a cryptographic fingerprint of data, a timestamp that cannot be backdated, and a public way to verify both, are what such infrastructure needs to show regulators and partners that data and policies were not altered after the fact.

Question 1: Exploring a cryptocurrency exchange

I explored Coinbase from Rwanda: I created an individual account, verified my identity, and looked through the dashboard, the trading area, the transfer options, the transaction history and the security settings, without buying or sending any cryptocurrency.

Step 1: creating an account. I signed up with my school email through Google, confirmed my phone number with a one-time code, confirmed that I am over 18, and chose Rwanda as my country. Coinbase then asked for a government ID (a passport or a national ID, front and back), and I used my national ID. It read my legal name, date of birth and ID number from the card, and finally asked what I would use Coinbase for, where my money comes from, my employment status, my expected yearly trading volume and my industry. Exchanges verify identity to stop money laundering, terrorist financing and fraud: if every account belongs to a known person, stolen or criminal money can be traced and suspicious activity reported. The international standard comes from the Financial Action Task Force (FATF), and in Rwanda, Law nº 023/2026 regulating virtual asset business requires licensed providers to have anti-money-laundering controls and to report suspicious transactions.





Step 2: exploring the platform. The dashboard has Home, Trade, Agents, Lend and Transactions in the side menu, and the trade page lists coins with their market price, 24-hour volume and market cap, shown in Rwandan francs. Bitcoin (BTC, about RWF 114.7 million) is the first cryptocurrency, used to send and store value; its supply is fixed at 21 million coins. Ethereum (ETH, about RWF 3.3 million) runs smart contracts and pays for using them; unlike Bitcoin it is programmable, which is what this project's contracts run on. Solana (SOL, about RWF 139,000) is built for speed and low fees: Coinbase's transfer screen estimated about 1 second for a Solana transfer against about 24 minutes for Bitcoin and 3 minutes for Ethereum.





Step 3: payment methods. Payment methods are added under Settings, Payment methods. A bank account needs the account holder's name, the bank and the account number; a debit card needs the card number, expiry date and security code, and is checked with a small test charge. A bank transfer is usually the cheaper option but can take one or more working days, while a debit card is instant but charges a higher percentage fee. I did not add a payment method to my account.

Step 4: buying $100. A market order buys straight away at the best price available, so it always goes through but the price can move slightly while it fills. A limit order only buys if the price reaches a level I set, so the price is guaranteed but the purchase is not. For a small first purchase I would choose a market order, because speed matters more than a small price difference. The amount received is ($100 minus Coinbase's fee) divided by the price of one coin; Coinbase shows both on the order preview before confirming. I did not place or preview an order.

Step 5: the transaction record. My Transactions page was empty, but it can be filtered by product, activity, status, asset and date, and each entry records the type, asset, amount, value, fee, status and time. A transaction hash is the unique ID of a transaction sent on a blockchain. Anyone can paste it into a block explorer to prove that the transfer happened, when, between which addresses and for how much, so it is the receipt that support teams and recipients ask for.



Step 6: security features. Coinbase rated my security "Needs attention". 2-step verification asks for a second code, from my phone or an authenticator app, on top of my password, so a stolen password is not enough to get in. Transfer protection adds checks before crypto leaves the account, for example confirming new addresses, which stops an attacker who gets in from emptying it. The page also lists active sessions and confirmed devices that I can sign out, and a button to lock the account in an emergency.



Blockchain setup

The project uses Hardhat 2.29.1, a development environment for Ethereum written in JavaScript, with Node.js 24. The setup steps were:

1. Install Node.js, which runs Hardhat and the scripts.
2. Create the project folder and install Hardhat, the Hardhat toolbox (which bundles ethers.js for talking to contracts, and Mocha and Chai for tests) and OpenZeppelin Contracts, the audited library the contracts build on.
3. Configure the compiler in hardhat.config.js: Solidity 0.8.28 with the optimizer on and the Cancun EVM version.
4. Start a local node with npx hardhat node.

The local node is a full Ethereum blockchain running only on this computer. It mines a block for every transaction instantly and starts with 20 accounts, each holding 10,000 test ETH, whose addresses and private keys it prints at startup.

```
pc@desktop-unknown:~/Documents/alu-dapps/proj_1_assgmnt/q2_alu_logo_registry$ npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
========

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000 ETH)
Private Key: 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba

Account #6: 0x976EA74026E726554dB657fA54763abd0C3a0aa9 (10000 ETH)
Private Key: 0x92db14e403b83dfe3df233f83dfa3a0d7096f21ca9b0d6d6b8d88b2b4ec1564e

Account #7: 0x14dC79964da2C08b23698B3D3cc7Ca32193d9955 (10000 ETH)
Private Key: 0x4bbbf85ce3377467afe5d46f804f221813b2bb87f24d81f60f1fcdbf7cbf4356

Account #8: 0x23618e81E3f5cdF7f54C3d65f7FBc0aBf5B21E8f (10000 ETH)
Private Key: 0xdbda1821b80551c9d65939329250298aa3472ba22feea921c0cf5d620ea67b97

Account #9: 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 (10000 ETH)
Private Key: 0x2a871d0798f97d79848a013d4936a73bf4cc922c825d33c1cf7073dff6d409c6

Account #10: 0xBcd4042DE499D14e55001CcbB24a551F3b954096 (10000 ETH)
Private Key: 0xf214f2b2cd398c806f84e317254e0f0b801d0643303237d97a22a48e01628897

Account #11: 0x71bE63f3384f5fb98995898A86B02Fb2426c5788 (10000 ETH)
Private Key: 0x701b615bbdfb9de65240bc28bd21bbc0d996645a3dd57e7b12bc2bdf6f192c82
```

What is a local blockchain, and why use one? A local blockchain behaves like Ethereum (same accounts, transactions, gas and contract execution) but exists only on the developer's machine. Developers use it because every transaction is free and confirmed instantly, the chain can be reset at any time, and mistakes cost nothing. On the real network, every deployment and test transaction costs real money, waits for a block, and stays on the public record forever. Here, the contracts were developed and tested locally first, and only then deployed to the Sepolia test network.

What is a wallet address? A wallet address is a public identifier on Ethereum, 20 bytes written as 42 hexadecimal characters starting with 0x. It is derived from the public half of a key pair: the

last 20 bytes of the Keccak-256 hash of the public key. Whoever holds the matching private key controls the address and is the only one who can sign transactions from it. On the blockchain an address represents an account: it holds a balance of ETH and tokens, owns NFTs, and is recorded as the sender of every transaction it signs. In this project the address that registers the logo is stored as its registrant, and the address that receives the ALUT supply is the logo's owner.

Smart contract explanation

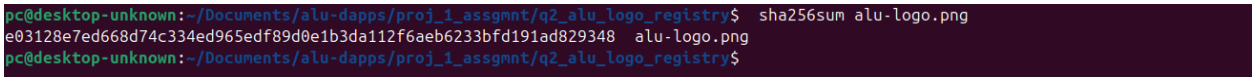
What a smart contract is

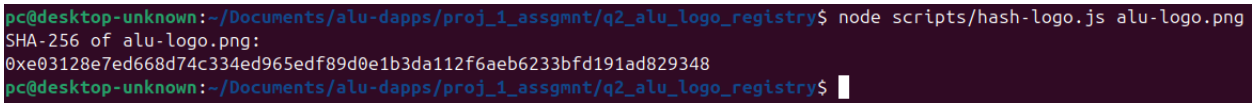
A smart contract is a program stored on a blockchain at its own address. Its code and data are copied to every node, and every node runs it and must reach the same result, so no single party can change what it does or what it stored. A normal program on a server is controlled by whoever runs the server: it can be edited, switched off, or have its database changed, and users must trust the operator. A deployed smart contract cannot be edited, keeps running as long as the network does, and its rules apply equally to everyone, including the person who deployed it.

Generating the logo hash

The ALU logo was downloaded as a WebP image saved with a .png name. It was converted losslessly to a real PNG (every pixel identical) so that the file type recorded on chain is true. The SHA-256 hash of that PNG was generated with a small Node.js script using the built-in crypto module, and confirmed with the sha256sum command:

```
None
0xe03128e7ed668d74c334ed965edf89d0e1b3da112f6aeb6233bfd191ad829348
```





The deployment script hashes the file itself when it runs, so the value registered on chain always comes from the real file and can never be a mistyped placeholder.

ALUAssetRegistry

The contract extends OpenZeppelin's ERC-721. Each registered file is described by an AssetMetadata struct holding the asset name, the file type, the SHA-256 hash as a bytes32 value, the address that registered it and the Unix timestamp of registration. Two mappings hold

the data: one from token ID to its AssetMetadata, readable by anyone, and one from content hash to a true or false value that records whether the hash has already been registered.

registerAsset(name, fileType, contentHash) registers a new file. It first refuses empty values and any hash that is already registered, reverting with the message "This file has already been registered". It then increases the token counter, marks the hash as used, saves the metadata with the caller's address and the current block time, mints the new NFT to the caller with _safeMint, emits an AssetRegistered event, and returns the new token ID. The deployment script calls it with "ALU Official Logo", "image/png" and the logo's hash, and the logo becomes token 1.

verifyLogoIntegrity(tokenId, contentHash) compares the hash stored for a token with a hash supplied by the caller. If they match it returns true and "Logo is authentic."; otherwise it returns false and "Warning: logo does not match." It is a view function, so it only reads data and costs no gas when called from outside the chain.

getAsset(tokenId) returns the full AssetMetadata of a token, so anyone can see what was registered, by whom and when. Both view functions refuse a token ID that was never registered, rather than answering with empty data. A small extra view, totalAssets, returns how many assets exist.

Security measures. The contract builds on audited OpenZeppelin code instead of reimplementing ERC-721. It validates every input. It saves the metadata and marks the hash as used before calling _safeMint, because _safeMint hands control to the receiving contract (to confirm it can hold NFTs); doing it in this order, known as checks-effects-interactions, means that outside code can never register the same hash twice. The deployment script registers the logo in the same run that deploys the registry, so nobody can register the logo's hash first.

Why an NFT, and why ERC-721

An NFT (non-fungible token) is a token that is unique: each one has its own ID and is not interchangeable with any other, unlike coins where any one unit equals another. ERC-721 is the Ethereum standard for such tokens. It fits the logo because there is exactly one official logo: it needs one unique token with one owner, a standard way to look up that owner and transfer the token, and compatibility with wallets, explorers and marketplaces that already understand ERC-721.

SHA-256 and why one pixel changes everything

SHA-256 is a cryptographic hash function: it turns input of any size into a fixed 256-bit (32-byte) value. The same file always gives the same hash, but it is practically impossible to find a different file with the same hash or to work back from the hash to the file. SHA-256 is designed so that changing a single bit of the input changes, on average, half of the output bits in an unpredictable way (the avalanche effect). Editing one pixel changes some bytes of the file,

which changes the hash completely. One of the tests changes a single byte of the logo and confirms that verifyLogoIntegrity then answers "Warning: logo does not match."

Why stored data cannot be changed

Every block includes the hash of the block before it, so altering an old transaction would change that block's hash and break every block after it. Thousands of independent nodes hold their own copy and reject any chain that does not follow the rules. Once a block is finalised, replacing it would require validators holding a large share of all staked ETH to sign conflicting blocks, and the protocol destroys (slashes) the stake of validators caught doing that. The contract itself also offers no function that edits or deletes a registration: the only thing that can happen to a hash is being registered once.

Tokenisation

What tokenisation means

Tokenisation means representing ownership of an asset as digital tokens on a blockchain, so that ownership can be split, transferred and checked without a central register. Instead of ALU alone owning the whole logo, ownership is divided into 1,000,000 equal shares, and each share is one ALUT token.

ALULogoToken

The contract extends OpenZeppelin's ERC-20 and Ownable. Its constructor takes the logo owner's address, mints all 1,000,000 ALUT to it, and makes it the contract owner. The token uses 0 decimals, so one token is exactly one share and amounts are whole numbers.

distributeShares(recipient, amount) can only be called by the owner (onlyOwner). It requires an amount greater than zero and a real recipient address, then moves that many shares from the owner's balance to the recipient and emits a SharesDistributed event. It fails if the owner holds fewer shares than requested.

ownershipPercentage(holder) returns the holder's balance multiplied by 100 and divided by the total supply, as a whole number rounded down. 500,000 ALUT gives 50.

Who at ALU would hold ALUT

Holder	**Why**
ALU administration	keeps the largest share as the logo's original owner and the body responsible for the brand

Holder	Why
Faculty	recognises the staff whose work builds the reputation the logo stands for
Students and alumni	gives the community that carries the brand a visible stake in it
Partners	could hold shares under licensing or co-branding agreements

ERC-721 and ERC-20

ERC-721 tokens are unique and indivisible: token 1 is the logo and nothing else can stand in for it. ERC-20 tokens are fungible: every ALUT equals every other ALUT and they can be counted and split. The logo is registered as ERC-721 because it is one specific file that must be identified exactly; ownership shares are ERC-20 because they are interchangeable units that need to be divided among many holders in different amounts.

What 100,000 ALUT means

A faculty member holding 100,000 of 1,000,000 ALUT owns 10% of the logo, and ownershipPercentage returns 10. In practice this is a recorded, provable stake: the wallet can show its share at any time, transfer it or part of it, and, if ALU attached rights to the token such as a vote on how the logo may be licensed or a share of licensing income, those rights would be weighted at 10%. The shares do not change who registered the logo; the NFT still records ALU as the registrant.

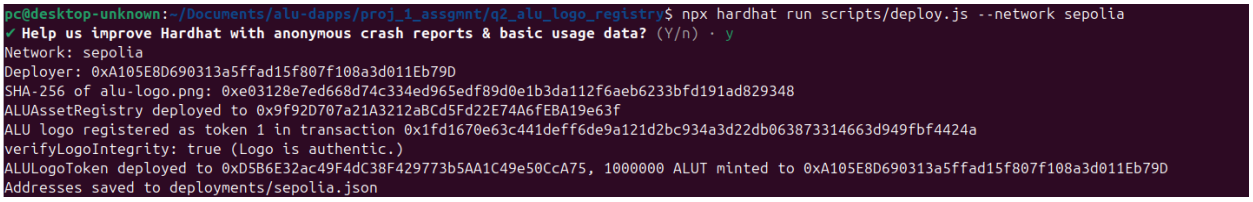
Testnet deployment

The contracts were deployed to the Sepolia test network through an Alchemy RPC endpoint and a MetaMask wallet funded with Sepolia ETH. The RPC URL and private key were stored with Hardhat's configuration variables, outside the project, so they never reached the repository.

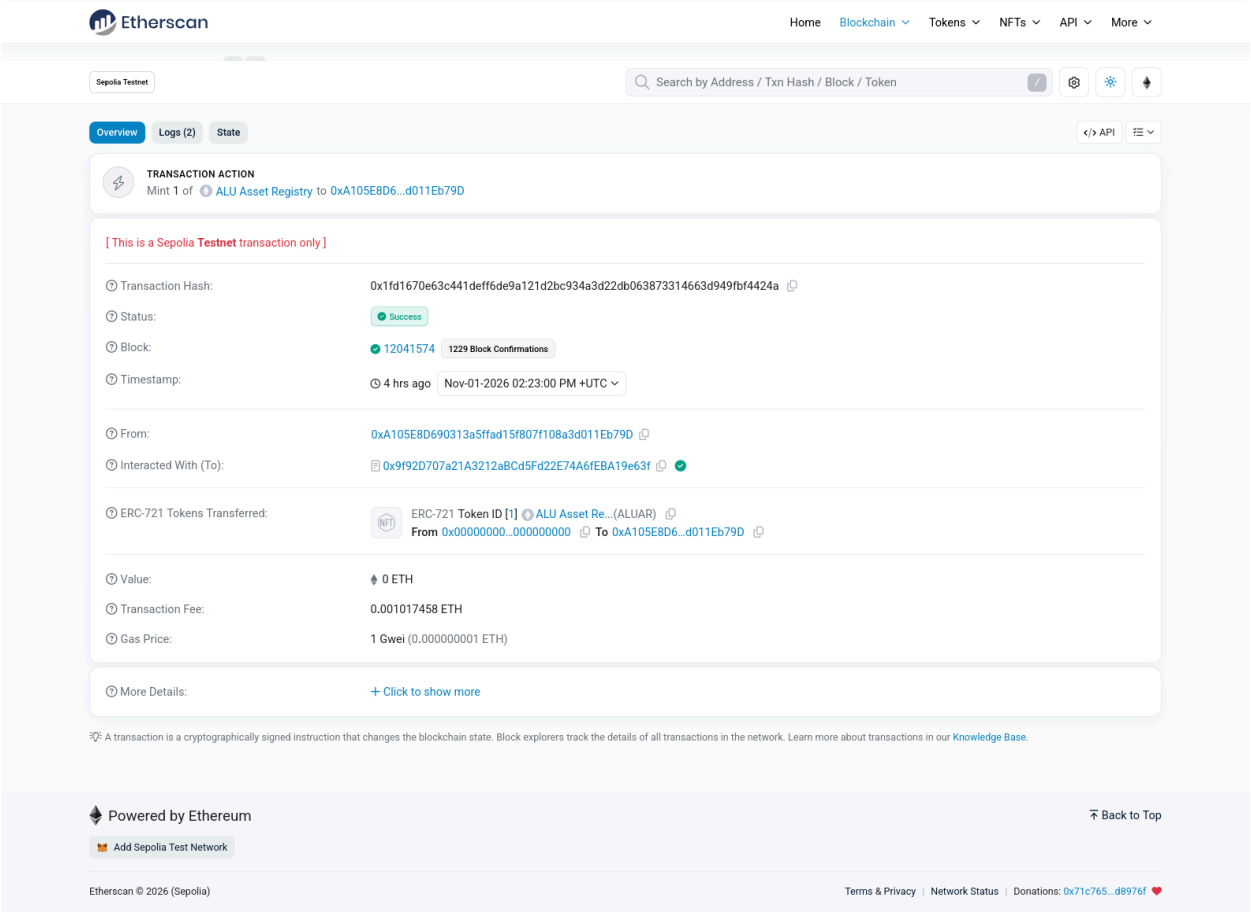
	Sepolia
ALUAssetRegistry	0x9f92D707a21A3212aBCd5Fd22E74A6fEBA19e63f
ALULogoToken	0xD5B6E32ac49F4dC38F429773b5AA1C49e50CcA75
Logo registration transaction	0x1fd1670e63c441deff6de9a121d2bc934a3d22db063873314663d949fbf4424a

	Sepolia
Deployer and logo owner	0xA105E8D690313a5ffad15f807f108a3d011Eb79D

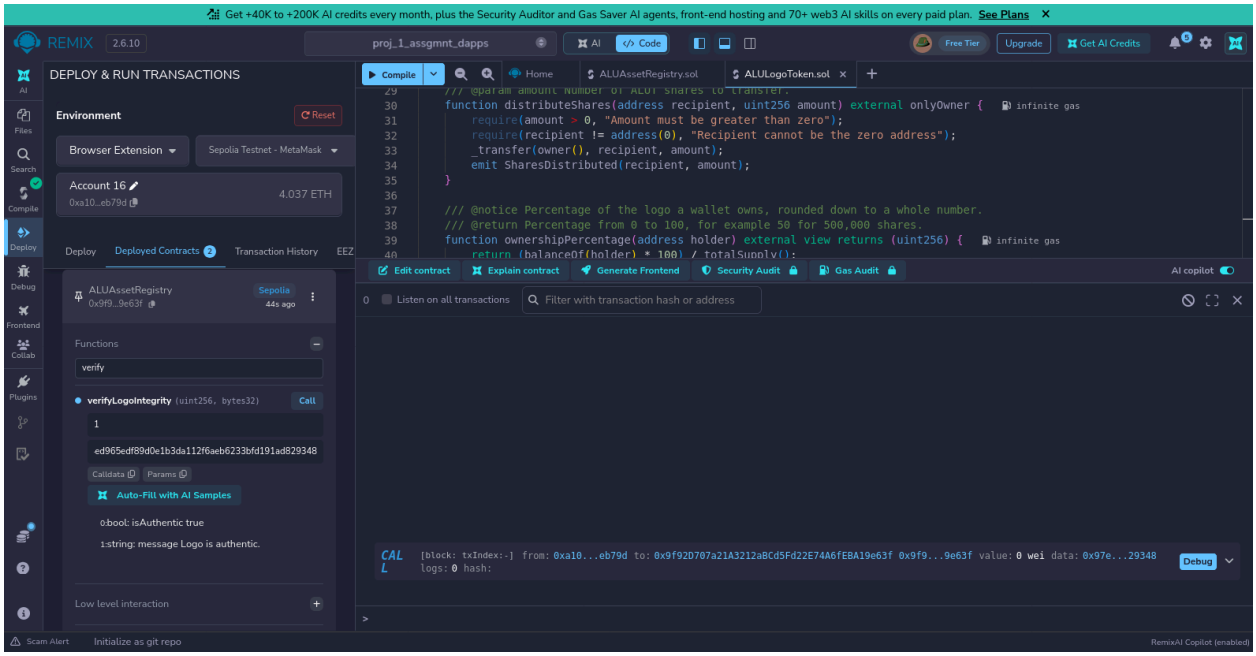
The contracts and the transaction can be inspected at https://sepolia.etherscan.io/address/0x9f92D707a21A3212aBCd5Fd22E74A6fEBA19e63f, https://sepolia.etherscan.io/address/0xD5B6E32ac49F4dC38F429773b5AA1C49e50CcA75 and https://sepolia.etherscan.io/tx/0x1fd1670e63c441deff6de9a121d2bc934a3d22db063873314663d949fbf4424a.

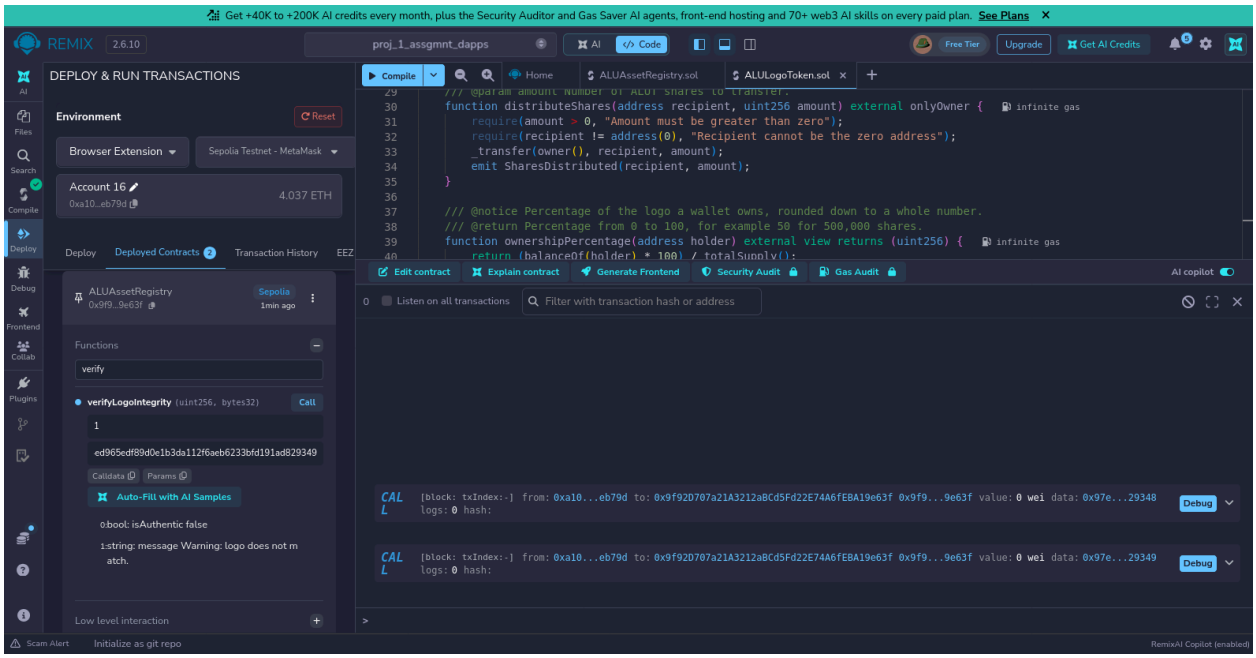




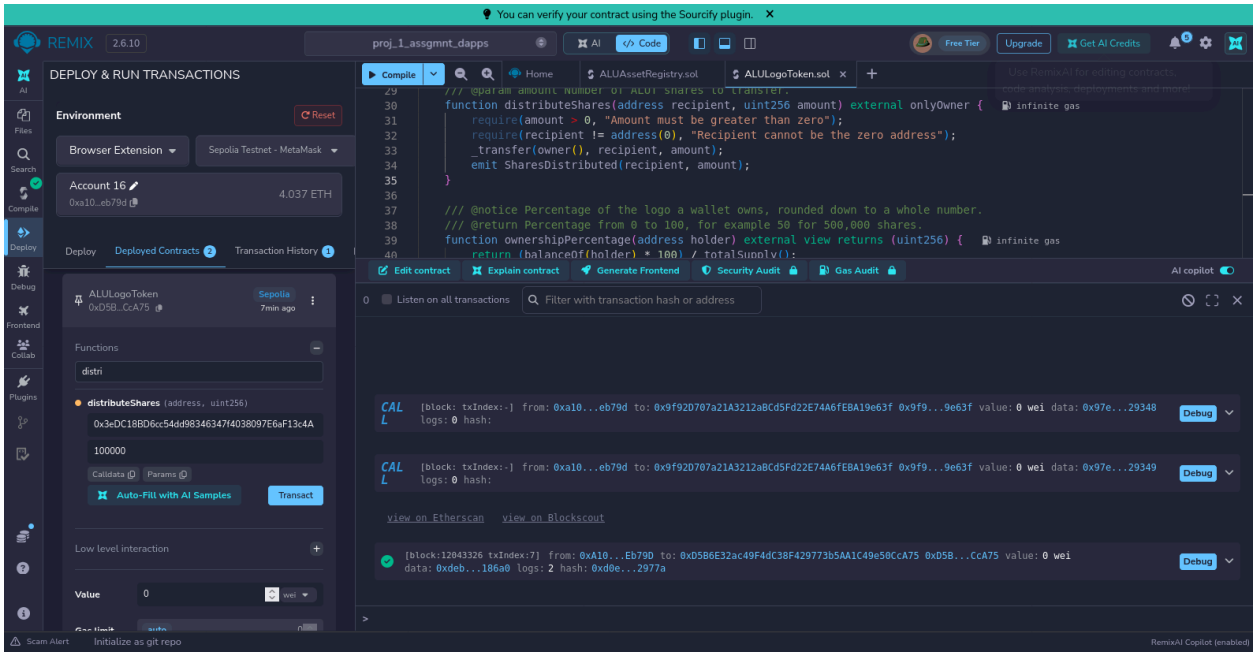


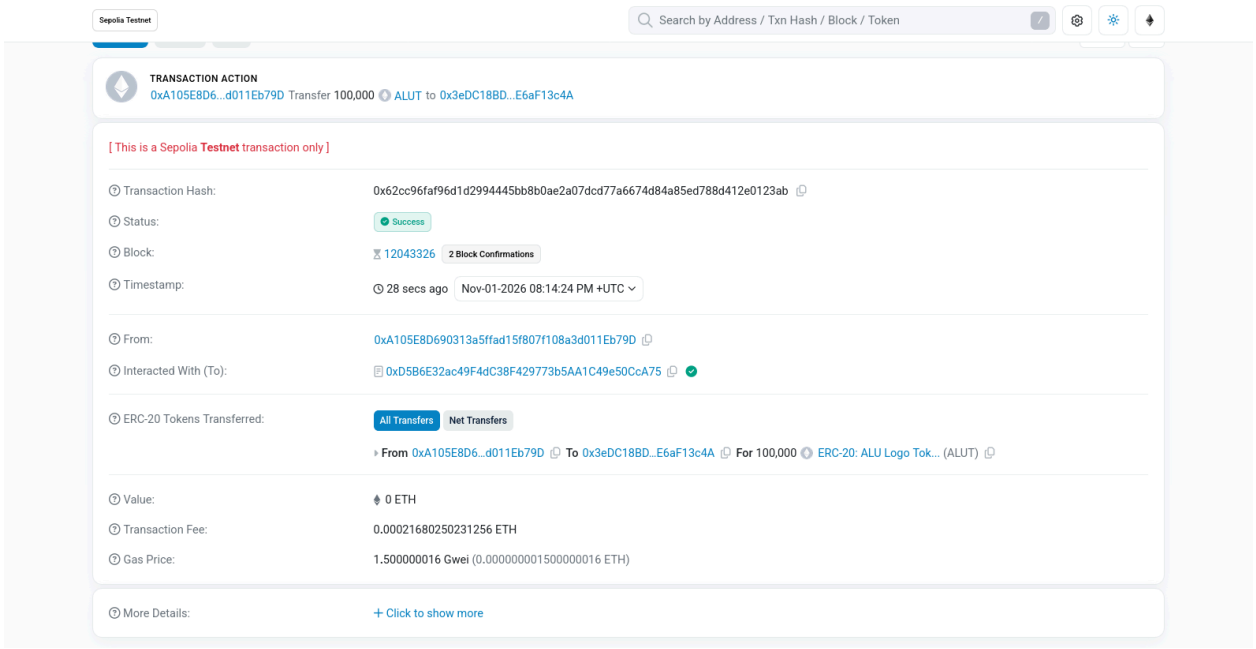
The logo registration was confirmed in block 12041574 and cost 0.001017458 Sepolia ETH at a gas price of 1 gwei; Etherscan shows it as the mint of ERC-721 token 1 to the deployer's address. The source code of both contracts was then verified on Etherscan, so anyone can read it next to the deployed bytecode and call the view functions from the browser. The deployed contracts were then loaded in Remix through MetaMask and tested on Sepolia: verifyLogoIntegrity with the real hash and with a wrong one, getAsset, distributeShares and ownershipPercentage.

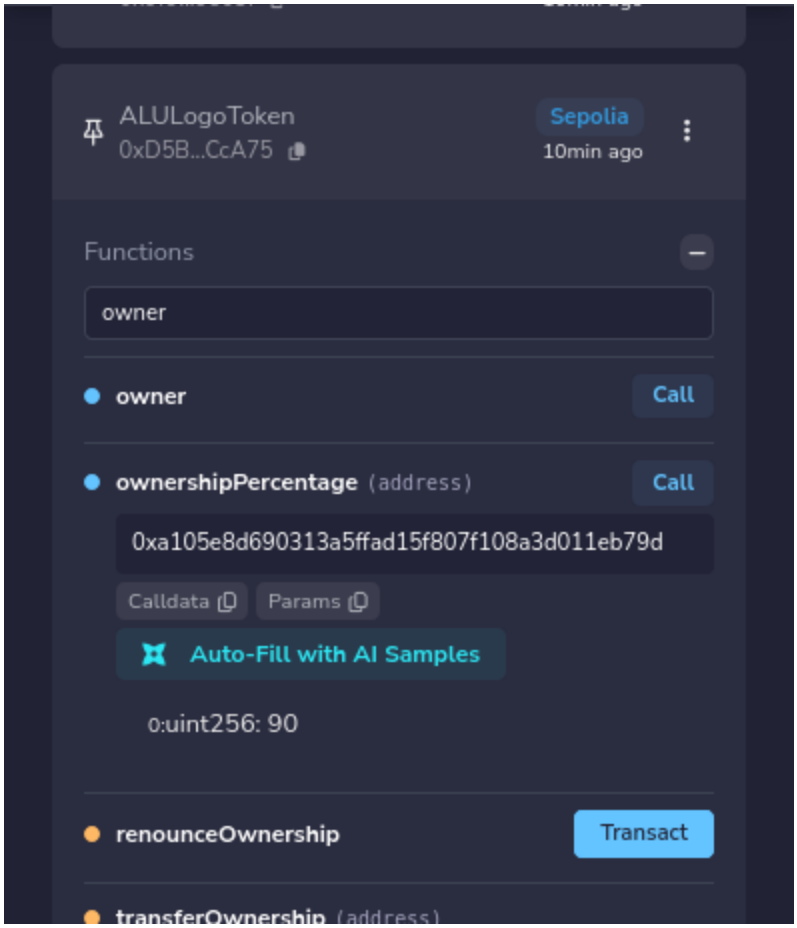


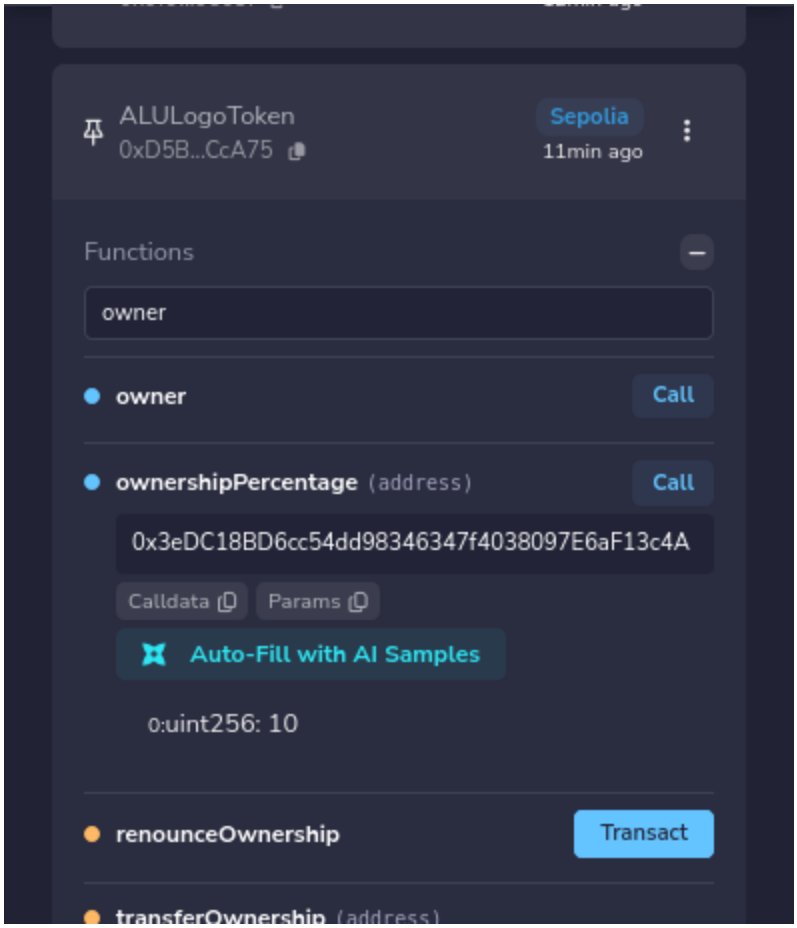


Shares were then distributed from Remix: distributeShares sent 100,000 ALUT from the logo owner to a second wallet (0x3eDC18BD6cc54dd98346347f4038097E6aF13c4A) in transaction 0x62cc96faf96d1d2994445bb8b0ae2a07dcd77a6674d84a85ed788d412e0123ab, confirmed in block 12043326 for a fee of 0.000217 Sepolia ETH. The transaction emitted both the standard ERC-20 Transfer event and the contract's SharesDistributed event. ownershipPercentage then returned 90 for the logo owner and 10 for the recipient.


You can verify your contract using the Sourcify plugin.
REMIX 2.6.10
proj_1_assgmnt_dapps
DEPLOY & RUN TRANSACTIONS
Environment
Reset
Browser Extension
Sepolia Testnet - MetaMask
Account 16
0xa10...eb79d
4.037 ETH
Deploy
Deployed Contracts 2
Transaction History 1
ALULogoToken
0xD5B...CcA75
Sepolia
7min ago
Functions
distri
distributeShares (address, uint256)
0x3eDC18BD6cc54dd98346347f4038097E6aF13c4A
100000
Calldata
Params
Auto-Fill with AI Samples
Transact
Low level interaction
Value
0
wei
Compile
Home
ALUAssetRegistry.sol
ALULogoToken.sol
Use RemixAI for editing contracts, code analysis, deployments and more!
/// @param amount Number of ALUT shares to transfer.
function distributeShares(address recipient, uint256 amount) external onlyOwner {
infinite gas
require(amount > 0, "Amount must be greater than zero");
require(recipient != address(0), "Recipient cannot be the zero address");
_transfer(owner(), recipient, amount);
emit SharesDistributed(recipient, amount);
}
/// @notice Percentage of the logo a wallet owns, rounded down to a whole number.
/// @return Percentage from 0 to 100, for example 50 for 500,000 shares.
function ownershipPercentage(address holder) external view returns (uint256) {
infinite gas
return (balanceOf(holder) * 100) / totalSupply();
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
CALL [block: txIndex:-] from: 0xa10...eb79d to: 0x9f92D707a21A3212aBCd5Fd22E74A6fEBA19e63f 0x9f9...9e63f value: 0 wei data: 0x97e...29348 logs: 0 hash:
Debug
CALL [block: txIndex:-] from: 0xa10...eb79d to: 0x9f92D707a21A3212aBCd5Fd22E74A6fEBA19e63f 0x9f9...9e63f value: 0 wei data: 0x97e...29349 logs: 0 hash:
Debug
view on Etherscan
view on Blockscout
[block:12043326 txIndex:7] from: 0xA10...Eb79D to: 0xD5B6E32ac49F4dC38F429773b5AA1C49e50CcA75 0xD5B...CcA75 value: 0 wei data: 0xdeb...186a0 logs: 2 hash: 0xd0e...2977a
Debug
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)


Sepolia Testnet
Search by Address / Txn Hash / Block / Token
TRANSACTION ACTION
0xA105E8D6...d011Eb79D Transfer 100,000 ALUT to 0x3eDC18BD...E6aF13c4A
[This is a Sepolia Testnet transaction only]
Transaction Hash:
0x62cc96faf96d1d2994445bb8b0ae2a07dcd77a6674d84a85ed788d412e0123ab
Status:
Success
Block:
12043326
2 Block Confirmations
Timestamp:
28 secs ago
Nov-01-2026 08:14:24 PM +UTC
From:
0xA105E8D690313a5ffad15f807f108a3d011Eb79D
Interacted With (To):
0xD5B6E32ac49F4dC38F429773b5AA1C49e50CcA75
ERC-20 Tokens Transferred:
All Transfers
Net Transfers
From 0xA105E8D6...d011Eb79D To 0x3eDC18BD...E6aF13c4A For 100,000 ERC-20: ALU Logo Tok... (ALUT)
Value:
0 ETH
Transaction Fee:
0.00021680250231256 ETH
Gas Price:
1.500000016 Gwei (0.000000001500000016 ETH)
More Details:
+ Click to show more


ALULogoToken
0xD5B...CcA75
Sepolia
10min ago
Functions
owner
owner
Call
ownershipPercentage (address)
Call
0xa105e8d690313a5ffad15f807f108a3d011eb79d
Calldata
Params
Auto-Fill with AI Samples
0: uint256: 90
renounceOwnership
Transact
transferOwnership (address)



Test results

The test file contains the eight required tests, written with Mocha, Chai and ethers. Each runs on a fresh local chain.

Test	**Checks**
registers the ALU logo and returns a token ID	token ID 1, the AssetRegistered event, the caller owns the NFT
rejects a second registration of the same hash	revert with "This file has already been registered", even from another account
verifyLogoIntegrity returns true for the correct hash	true and "Logo is authentic."
verifyLogoIntegrity returns false for an incorrect hash	false and "Warning: logo does not match." for the logo with one byte changed

Test	Checks
getAsset returns the asset name and file type	name, file type, hash and registrant
mints the full supply of 1,000,000 ALUT to the logo owner	total supply and owner balance
distributeShares transfers the given number of tokens	owner balance down and recipient balance up by 100,000
ownershipPercentage returns the percentage held by a wallet	500,000 ALUT gives 50



Six further tests cover empty inputs, unknown token IDs, two different files, a non-owner calling distributeShares, distributing zero or too many shares, and rounding. All 14 pass.

Challenges

1. A new EVM instruction. The first compilation failed: OpenZeppelin 5.6 uses the mcopy instruction, which was added to Ethereum in the Cancun upgrade, while Hardhat 2 compiles for an older EVM version by default. Contracts are compiled for a specific version of the Ethereum Virtual Machine, and code that uses a newer instruction cannot run on a chain that does not have it. Setting the compiler's EVM version to Cancun solved it, after checking that Sepolia, the local node and Remix all support Cancun.

2. Control leaving the contract in the middle of a registration. _safeMint calls onERC721Received on the receiving address if it is a contract, which runs outside code before registerAsset finishes. If the hash were marked as used only after minting, that outside code could call registerAsset again with the same hash and register the logo twice. The fix was to follow the checks-effects-interactions pattern: check the inputs, save the metadata and mark the hash, and only then mint.

3. Token decimals. ERC-20 amounts are stored in the smallest unit, and tokens normally have 18 decimals, so a call to distribute 100,000 tokens would really move 0.0000000000001 ALUT.

Because ALUT represents shares of one logo, the token was given 0 decimals: one token is one share, and the numbers in distributeShares and ownershipPercentage mean exactly what they say.

Conclusion

The project showed how a blockchain turns a file into a verifiable asset: a 32-byte fingerprint, written once with a timestamp and an owner, is enough for anyone to tell the official logo from a copy without trusting a server. Building it showed why audited libraries, input checks and the order of state changes matter when code is permanent once deployed, and how a local chain makes it safe to get those details right before paying for a public deployment.

With more time, I would add:

- a tokenURI pointing to the logo stored on IPFS, so wallets can display it
- a link from ALULogoToken to the registry's token 1, so the shares are tied to the NFT on chain
- a two-step ownership transfer, so ALU's admin key cannot be handed to a wrong address by mistake
- the web interface planned for the next stage, where anyone can drop a file in the browser and see whether it is authentic

GitHub link

https://github.com/devark28/alu-dapp-proj_1_assgmnt

Video

https://youtu.be/KaoQenSY1VY